

P3 Member Playbook

Reliable Data Transfer over UDP · CS-30003 Coding Assignment 1 · Written 31 August 2026

How to use this document. Part 0 and Part 1 are for everyone — read them today. After that, find your own name and work only from your section. Each week tells you what to do, how to know you are finished, and what you must be able to say out loud about it. The last column matters most: the viva is worth 5 of the 10 marks and it is assessed *individually*, so finishing your code is only half of finishing your job.

Who is doing what

MEMBER	ROLE	OWNS	EXPERIMENT & REPORT SECTION
Shaili Seth 2405901	M1	Wire format, RFC 1071 checksum, metadata handshake, Stop-and-Wait, sender/receiver CLIs, FIN teardown	Exp 1 — goodput vs loss rate Report: wire format & session
Sinjan Mishra 2405910	M2	Timer heap, Jacobson/Karels RTO with Karn, Go-Back-N	Exp 4 — RTO sensitivity Report: timers, RTO, GBN
Sohini Pandit 2405913	M3	Selective Repeat, receiver buffer, selective ACKs, wraparound correctness, SACK stretch	Exp 2 — goodput vs window Report: Selective Repeat
Sohan Mandal 2405912	M4	Channel emulator, capacity calibration, experiment harness, statistics, plots, tc/netem validation	Exp 3 — reordering Report: methodology & emulator

The whole project on one page

WEEK	SHAILI (M1)	SINJAN (M2)	SOHINI (M3)	SOHAN (M4)
1 Sep 1–7	Rewrite Packet by hand; metadata handshake	Read RFC 6298 & GBN theory; design timer heap on paper	Read SR theory; work out the window bound; design receiver buffer	Rewrite emulator by hand; re-run calibration on WSL2
2 Sep 8–14	Stop-and-Wait; sender/receiver CLIs	Timerwheel + tests	Receiver buffer prototype; wraparound test cases	Emulator v1 complete; JSONL trace verified
3 Sep 15–21	FIN/FINACK teardown; harden S&W	GoBackN + RttEstimator	SelectiveRepeat core: send/receive windows	RunStats CSV + run_matrix.py ; first plot
4 Sep 22–28	Support; integrity suite across all three	Wire adaptive RTO into all three protocols	Finish SR + wraparound tests. Core complete	Support; prepare experiment configs
5 Sep 29–Oct 5	Experiment 1 ; methodology write-up	Support experiments; start report section	Experiment 2	Experiment 3 ; plots.py ; run the full matrix
6 Oct 6–12	Cross-teaching; report section	Experiment 4 with spurious labelling	Cross-teaching; report section	tc netem cross-validation; cross-teaching
7 Oct 13–19	Report review	Report section done	Stretch: SACK blocks	Final figures; report assembly
8 Oct 20–26	Code freeze 22 Oct · final report · viva drills — everyone			

Part 0 · Setup — everyone, before 1 September

Do this today or tomorrow. Nobody can start their week 1 without it, and the whole schedule assumes it is already done.

1. Install the tools

On Windows, install **JDK 21** (Adoptium Temurin) and **Git**. Then install a JDK inside WSL2 as well, because the experiments must run on Linux:

```
wsl --install -d Ubuntu          # only if you do not already have WSL2
wsl -d Ubuntu
sudo apt update && sudo apt install -y openjdk-21-jdk make python3-pip
java -version                   # must print 21.x
```

2. Clone the repository

```
git clone https://github.com/sohan1611/rdt-udp.git
cd rdt-udp
```

3. Set your git identity — this is the step people get wrong

Run this **inside the repo folder**, using the email address on your own GitHub account:

```
git config user.name "Your Full Name"
git config user.email "your-github-email@example.com"
git log -1 --format='%an <%ae>'      # check it took effect after your first commit
```

If this email is not on your GitHub account, you will not appear as a contributor. Your commits show a grey avatar with no profile link, and as far as the contributor graph is concerned you did nothing. Sir treats commit history as evidence in individual moderation, so this is worth thirty seconds of care. Find your address at github.com/settings/emails; if you would rather not publish a personal one, GitHub gives you a private `...@users.noreply.github.com` address on that same page that links correctly.

4. Check the build works

```
./build.sh          # or: make test
```

You should see three suites and **33 tests, 0 failed**. If not, say so in the group chat before 1 September rather than discovering it mid-week.

5. Read these, in this order

- `README.md` — what the project is and how to run it (10 minutes)
- `docs/PLAN.md` sections 1–4 — the design decisions we are committed to (30 minutes)
- `AI-USE.md` — the rules on AI, and the two open rewrite items
- Your own section of this document

Part 1 · Rules everyone follows

Branches and pull requests

Never commit straight to `main`. Branch per task, named `m1/packet-rewrite`, `m2/timer-wheel`, `m3/receiver-buffer`, `m4/emulator-rewrite` and so on.

```
git checkout main && git pull
git checkout -b m2/timer-wheel
# ... work, commit as you go ...
git push -u origin m2/timer-wheel
# then open the PR on GitHub
```

Review pairing (rotates weekly)

WEEKS	PAIRS
Odd (1,3,5,7)	Shaili ↔ Sinjan · Sohini ↔ Sohan
Even (2,4,6,8)	Shaili ↔ Sohini · Sinjan ↔ Sohan

The explanation check — do not skip this. Before approving any pull request, the reviewer asks the author two or three *why* questions about the code. *Why this data structure? What happens if this ACK is lost? Why this bound and not that one?* If the author cannot answer, the PR does not merge. It costs five minutes, it rehearses the viva every single week, and it catches copy-pasted code in September when there is still time to fix it rather than in the viva room when there is not.

Commits

- At least **three commits a week each**, small and frequent. A history of three commits in the final week is called out in the brief as a warning sign.
- Commit under your own name and email. Never commit on someone else's behalf — the brief counts that as non-contribution by the absent member.
- **No Co-Authored-By lines. No bot pull requests. Dependabot stays off.** The contributor list must show exactly our four names.

AI use

Permitted: explaining concepts, debugging, reviewing your code, generating tests, boilerplate, plotting scripts, improving your writing. **Not permitted: generating the core protocol implementation and submitting it as your work.** Log anything material in `AI-USE.md` the same day — honest disclosure carries no penalty, undisclosed use found at the viva does.

A chatbot asked to write Go-Back-N in Java gives everyone the same answer, and the brief treats identical-looking submissions across groups as collusion. Another group in our year is doing P3 too. Write your own code.

Banned libraries

No Netty, Aeron, KryoNet, JGroups, or any QUIC library. No TCP anywhere on the data path. `java.net.DatagramSocket` and the JDK standard library only. Python with pandas/matplotlib is fine, but for analysis only.

Shaili Seth · M1 · Framing and Session

You own the envelope every packet travels in, and the simplest of the three protocols. Your week 1 is the critical path: nobody can test anything until `Packet` is merged.

Week 1 · Sep 1–7

Rewrite `Packet.java` by hand · BLOCKS EVERYONE

Goal: a wire format you wrote yourself that passes all 13 existing tests.

1. Read `docs/PLAN.md` section 4.1 and study the 20-byte header diagram until you can draw it from memory.
2. Read **RFC 1071**, section 1. It is about one page and describes the checksum.
3. `git checkout -b m1/packet-rewrite`
4. Open `src/main/java/rdt/Packet.java`. Read it once to see the shape, then **delete the bodies** of `encode()`, `decode()` and `checksum()` and write them yourself from the header diagram. If you cannot write them without looking, you do not understand them yet — that is the signal to go back to the diagram, not to copy.
5. Three traps that will cost you a day each if you miss them:
 - `seq` and `ack` are **unsigned** 32-bit on the wire. A Java `int` is signed. Read them with `Integer.toUnsignedLong(buf.getInt())` into a `long`.
 - The 16-bit fields need `Short.toUnsignedInt(buf.getShort())`.
 - The checksum is computed with its own field set to **zero**, then written into that field. Verifying works the same way in reverse.
6. Run `./build.sh` until `PacketTest` reports 13 of 13.
7. Add the **metadata handshake**: a `seq=0` DATA packet whose payload carries the filename, the file size and the SHA-256 hash. The receiver parses it and replies with ACK 0. Real data then starts at `seq=1`.
8. Open the PR. Sinjan reviews and asks you three *why* questions.
9. Tick your box in `AI-USE.md`, sign it and date it.

Done when: `./build.sh` prints `PacketTest: 13 run, 0 failed`, the PR is merged into `main`, and the `AI-USE.md` box for `Packet.java` is ticked.

Be able to explain: why the checksum uses one's complement rather than a plain sum · what end-around carry does and why it matters · why a single-bit flip is *always* detected · what visibly breaks if you forget `Integer.toUnsignedLong` · why the checksum field must be zeroed first.

Week 2 · Sep 8–14

Stop-and-Wait and the CLIs

Goal: an 8 MiB file crosses the emulated network and the hash matches.

1. Write `StopAndWait.java` against the `ArqProtocol` interface. Sender: send one packet, start a timer, wait for its ACK, retransmit on timeout, then move on. Receiver: verify the checksum, ACK, deliver in order, silently discard duplicates.
2. Use **alternating-bit** sequencing (0, 1, 0, 1...) and be ready to explain why one bit is enough here but not for Go-Back-N.
3. Write `app/Sender.java` and `app/Receiver.java`: take a file, a host, a port and a `--protocol` flag.
4. Test against Sohan's emulator, first at 0% loss, then at 10%:

```
java -cp build/classes emulator.NetEm --listen 9000 --to 127.0.0.1:9001 \
  --seed 1 --both loss=0.10,delay=20,jitter=5 &
java -cp build/classes app.Receiver --port 9001 --out received.bin &
java -cp build/classes app.Sender --file test8m.bin --to 127.0.0.1:9000 --protocol stopwait
sha256sum test8m.bin received.bin      # the two hashes must match
```

Done when: the SHA-256 of the sent and received file match at 0% loss and again at 10% loss, across 10 different seeds.

Be able to explain: what happens when an ACK is lost but the data arrived · why the receiver must discard duplicates rather than deliver them · why Stop-and-Wait throughput is capped at MSS/RTT no matter how fast the link is.

Week 3 · Sep 15–21

Teardown and hardening

1. Add **FIN / FINACK** teardown so a transfer ends cleanly instead of the receiver guessing from silence.
2. Add the SHA-256 integrity check at the end of every transfer, comparing against the hash from the week 1 handshake.
3. Help Sinjan integrate Go-Back-N against your packet layer — expect one or two interface issues to surface here, and fix them now rather than in week 5.

Done when: a transfer ends with an explicit FIN exchange and prints a hash-match line, with no fixed sleep or timeout hack anywhere in the shutdown path.

Week 4 · Sep 22–28

Shared integrity suite

1. Build one integrity test that runs **all three** protocols over several channel profiles and several seeds, asserting a hash match every time.
2. This is the suite the week 4 gate is judged on, so it needs to be running before Sohini finishes Selective Repeat, not after.

Done when: the suite runs 3 protocols × 5 channel profiles × 10 seeds and every transfer matches.

Weeks 5–8

Experiment 1, report, viva

1. **Week 5:** run Experiment 1 — goodput against loss rate for all three protocols at window 32, ten seeds per point. Write the methodology section with Sohan.
2. **Week 6:** teach your module to the other three in the cross-teaching session, and learn theirs. Draft your report section on the wire format and session layer.
3. **Week 7:** review the full report draft; check every figure caption says what the reader should notice.
4. **Week 8:** viva drills. Code freeze 22 October.

Sinjan Mishra · M2 · Timers and Go-Back-N

You own everything to do with time: when to give up waiting, how long to wait, and what to resend. The hardest idea in your module is Karn's algorithm, and it is worth understanding properly because it is a favourite viva question.

Week 1 · Sep 1–7

Reading and design — no code yet

Goal: understand what you are building before Packet.java lands.

1. You are blocked on Shaili this week, so use it to read rather than to wait.
2. Read **RFC 6298** (computing TCP's retransmission timer) — it is short and it is exactly the Jacobson/Karels algorithm you will implement.
3. Read the Go-Back-N section of Kurose & Ross, chapter 3.
4. Write a one-page design note in `docs/` covering: what a timer heap is, why we use a single thread rather than a timer per packet, and how the socket receive timeout gets derived from the earliest deadline. Commit it — it counts as a commit, and it becomes part of your report section.
5. Review Shaili's `Packet` PR and ask three *why* questions.

Done when: the design note is committed and you can explain, without notes, why we avoid `ScheduledExecutorService` here.

Be able to explain: what SRTT and RTTVAR are · why RTO is $SRTT + 4 \times RTTVAR$ and not just SRTT.

Week 2 · Sep 8–14

TimerWheel

1. Write `TimerWheel.java`: a `PriorityQueue` of deadlines taken from `System.nanoTime()`, with operations to schedule, cancel, and ask for the earliest deadline.
2. It must be **single-threaded**. The socket loop asks the heap how long until the next timer, and blocks in `receive` for exactly that long. A timeout and an arriving ACK are then both just events on one loop, and there is no race between them.
3. Do **not** use `ScheduledExecutorService` or `java.util.Timer`. They work, but they reintroduce exactly the concurrency the brief warns about and they are much harder to defend.
4. Write tests: ordering under many insertions, cancelling a timer that already fired, two timers with identical deadlines.

Done when: your timer tests pass and the heap correctly reports the earliest deadline after a randomised sequence of schedules and cancels.

Be able to explain: how you avoid a race between a timeout firing and an ACK arriving — the answer is that there is no race, and you should be able to say exactly why.

Week 3 · Sep 15–21

Go-Back-N and the RTT estimator

1. Write `GoBackN.java`: a sliding send window of N packets, cumulative ACKs, one timer on the **base** of the window. On timeout, retransmit the base and everything after it.
2. Write `RttEstimator.java` implementing Jacobson/Karels: $SRTT = (1-\alpha) \cdot SRTT + \alpha \cdot R$ with $\alpha = 1/8$, $RTTVAR = (1-\beta) \cdot RTTVAR + \beta \cdot |SRTT - R|$ with $\beta = 1/4$, and $RTO = SRTT + 4 \cdot RTTVAR$, clamped to a sensible minimum.
3. Add **Karn's algorithm**: never take an RTT sample from a packet that was retransmitted. When an ACK arrives for a packet sent twice, you cannot tell which copy it answers, so using it would corrupt the estimate. On timeout, double the RTO instead (exponential backoff) and resume sampling only after a clean, un-retransmitted exchange.
4. Test the estimator against values you compute by hand on paper. This is the fastest way to catch an α/β mix-up.

Done when: Go-Back-N passes the integrity suite across 5 channel profiles $\times$ 10 seeds, and the estimator matches your hand-worked numbers.

Be able to explain: what Karn's algorithm protects against, with a concrete example · why GBN retransmits the whole window · why GBN needs $W \leq 2^k - 1$ sequence numbers.

Week 4 · Sep 22–28

Adaptive RTO everywhere

1. Wire `RttEstimator` into all three protocols, so Stop-and-Wait and Selective Repeat use adaptive RTO too.
2. Add a switch for a **fixed** RTO, so week 6 can compare fixed against adaptive. Support multipliers of 1.2, 1.5, 2 and 3 times the base RTT.

Done when: every protocol runs with either `--rto adaptive` or `--rto fixed:1.5`.

Weeks 5–8

Experiment 4, report, viva

1. **Week 5:** support the experiment runs; start drafting your report section.
2. **Week 6:** run **Experiment 4** — RTO sensitivity. Use Sohan's JSONL trace as ground truth to label each retransmission *spurious* (the original did arrive) or *necessary*. This is the experiment no other group will have; make the most of it.
3. **Week 7:** finish your report section on timers, RTO and Go-Back-N.
4. **Week 8:** viva drills. Code freeze 22 October.

Sohini Pandit · M3 · Selective Repeat

You own the hardest module in the project. That is why it is the only one you own, and why your reading starts in week 1 even though your code does not land until week 4. Do not leave the thinking until the build week.

Week 1 · Sep 1–7

Theory and the window bound

Goal: know exactly why Selective Repeat is harder than Go-Back-N.

1. Read the Selective Repeat section of Kurose & Ross, chapter 3, including the worked example where sender and receiver windows disagree.
2. Work out on paper why Selective Repeat needs $W \leq 2^{k-1}$ while Go-Back-N only needs $W \leq 2^k - 1$. Draw the failure case: a small sequence space, a full window, all ACKs lost, and a retransmission that the receiver mistakes for new data.
3. Write that explanation up as a short design note in `docs/` and commit it. It becomes the opening of your report section, and it is the single most likely thing you will be asked at the viva.
4. Review Sohan's emulator PR and ask three *why* questions.

Done when: the design note is committed and you can draw the window-overlap failure case on a whiteboard from memory.

Be able to explain: the window bound, with the concrete failure case — not just the formula.

Week 2 · Sep 8–14

Receiver buffer prototype

1. Build the **receiver buffer** on its own, before wiring it to any network code: a structure that accepts packets in any order, holds the out-of-order ones, and releases a contiguous run in order as soon as the gap fills.
2. Test it as a pure data structure — feed it sequences like 1, 3, 4, 2 and assert it emits 1, then nothing, then nothing, then 2, 3, 4 together.
3. Write the sequence-space wraparound test cases now, while the ideas are fresh. Make the sequence space size configurable so a test can force a wrap with a tiny space.

Done when: the buffer passes out-of-order, duplicate, and wraparound tests as a standalone unit, with no sockets involved.

Week 3 · Sep 15–21

SelectiveRepeat core

1. Write `SelectiveRepeat.java`: a sender window where **each unacknowledged packet has its own timer**, and a receiver that buffers out-of-order arrivals and acknowledges each packet individually.
2. On timeout, retransmit **only** that packet — this is the whole point of the protocol, and the reason it beats Go-Back-N under loss.
3. Encode selective ACKs using the `sackCount` field already reserved in the header, so no wire-format change is needed.
4. Use Sinjan's `TimerWheel` rather than writing your own timing.

Done when: a file transfers correctly at 10% loss and you can see in the logs that only the lost packets were resent, not whole windows.

Week 4 · Sep 22–28

Finish — this is the project gate

1. Run the wraparound tests you wrote in week 2 and make them pass.
2. Prove the window bound experimentally: with a small sequence space, show that a **legal** window transfers correctly and an **illegal** one corrupts the file. Most groups never test this; it is strong evidence and strong viva material.
3. Pass Shaili's shared integrity suite alongside the other two protocols.

Done when: all three protocols pass one shared integrity suite. This is the **Core complete** milestone for the whole project.

Fallback, decided 25 September — not in October. If Selective Repeat is not passing integrity tests by 25 September, ship it with a **single timer on the oldest unacknowledged packet** instead of one timer per packet. It is still correct and still beats Go-Back-N; per-packet timers move to the stretch list and the simplification gets documented in the report. Taking this decision on time is not failure — leaving it until October is.

Weeks 5–8

Experiment 2, stretch, report, viva

1. **Week 5:** run **Experiment 2** — goodput against window size (1 to 128) for Go-Back-N and Selective Repeat, at 1% and 5% loss. This is the experiment that shows the gap widening, which is the core of our claim.
2. **Week 6:** cross-teaching — teach Selective Repeat to the other three. Draft your report section.
3. **Week 7:** stretch goal — SACK blocks in the live protocol. Attempt this *only* once Core is solid.
4. **Week 8:** viva drills. Code freeze 22 October.

Sohan Mandal · M4 · Channel and Evidence

You own two things: the fake bad network everyone tests against, and all the measurement that turns working code into marks. Your weeks 1–2 unblock everybody, and your weeks 5–6 produce most of the report.

Week 1 · Sep 1–7

Rewrite the emulator by hand

Goal: an emulator you wrote yourself that passes all 20 existing tests.

1. The catalogue entry does not merely permit us to write the emulator, it requires it, in these words: "*a channel emulator **you write yourself***." The current draft is AI-written, so it must be replaced by hand. The 20 tests stay — generating tests is permitted, and they are what tell you the rewrite is correct.
2. Rewrite in this order, easiest first:
 1. `ChannelConfig.java` — the config parser. Trivial, gets you moving.
 2. `TraceLog.java` — the JSONL writer. Small and self-contained.
 3. `Channel.java` — the impairment model. The real one.
 4. `NetEm.java` — the socket loop and release queue.
3. Three design decisions to understand before you rewrite, because you will defend them:
 - **Separate process, not an in-process hook.** The protocol then has no test-only branches and genuinely does not know it is being tested.
 - `java.util.Random(seed)`. Its algorithm is fixed by the Java specification, so a seed reproduces byte-identically on any machine. Never `ThreadLocalRandom` or `SecureRandom` — neither guarantees that.
 - **A fixed number of random draws per packet**, needed or not. This means changing the loss rate does not shift the delay sequence, so two cells of a sweep sharing a seed differ only in the variable under test. It measurably lowers variance between neighbouring points on a curve.
4. Re-run the capacity calibration **inside WSL2** and commit that number, replacing the Windows one. The experiments run on Linux, so that is the host whose ceiling matters.
5. Tick your box in `AI-USE.md`, sign and date it.

```
./build.sh                                # 20 emulator tests must pass
java -Xms512m -Xmx512m -cp build/classes emulator.Calibrate
git add results/calibration.csv
```

Done when: `ChannelTest` and `NetEmSmokeTest` pass with your own code, `results/calibration.csv` holds a WSL2 measurement, and the `AI-USE.md` box is ticked.

Be able to explain: why the emulator is a separate process · why `java.util.Random` and not the alternatives · what the fixed draw count buys you · why we publish a capacity ceiling at all.

Week 2 · Sep 8–14

Emulator v1 complete

1. Confirm all five impairments work together: loss, duplication, corruption, reordering, and delay with jitter.
2. Verify the JSONL trace is complete and parseable after a hard kill — it flushes every 256 records, so a killed run loses at most that many lines.
3. Give Shaili a working emulator to test Stop-and-Wait against. She is blocked on you this week.

Done when: a 100,000-packet run reports measured loss, duplication and corruption rates within tolerance of what was configured, and the trace parses with zero malformed lines.

Week 3 · Sep 15–21

The evidence pipeline

1. Write `RunStats.java`: per-run counters — goodput, wire throughput, retransmissions, timeouts, duplicate ACKs — emitted as one CSV row.
2. Write `analysis/run_matrix.py`: reads a config, runs the sweep across parameters and seeds, and collects rows into `results/*.csv`.
3. Produce the **first real plot** from committed CSV data, even if it only has two protocols on it. Getting the pipeline end-to-end early is worth more than getting it pretty.

Done when: one command runs a sweep and a second command turns the resulting CSV into a PNG, with no manual steps in between.

Week 4 · Sep 22–28

Prepare and support

1. Lock the experiment configurations in `analysis/configs/` so runs are reproducible: 8 MiB file, 1400-byte payload, 10 seeds per cell.
2. Check every planned operating point sits below **half** the calibrated ceiling.
3. Help Sohini get Selective Repeat over the line — week 4 is the project gate and she carries the hardest module.

Week 5 · Sep 29–Oct 5

Run the matrix

1. Run Experiments 1, 2 and 3 at full seed count. Expect this to take hours, so start it early in the week, not on the Friday.
2. Write `analysis/plots.py`: every figure generated by script from committed CSVs. No hand-made plots anywhere in the report.
3. Report mean with **95% confidence intervals** over seeds on every point.
4. Run **Experiment 3** yourself — retransmission count against reordering probability.

Done when: figures 1–3 regenerate from committed CSVs by running one command.

Weeks 6–8

Validation, report, viva

1. **Week 6:** the `tc netem` cross-validation. Re-run the loss sweep using Linux's own emulator instead of ours and show the goodput curves agree within confidence intervals. Report any divergence honestly and explain it — netem's loss model is not identical to ours, and knowing that is the point. This pre-empts the sharpest question available at the viva.
2. **Week 6:** run the cross-teaching session. You must be able to explain Selective Repeat even though Sohini wrote it.
3. **Week 7:** final figures; assemble the report from everyone's sections.
4. **Week 8:** confirm `make all` reproduces every figure from a clean clone. Viva drills. Code freeze 22 October.

Part 6 · Shared checkpoints

Every week ends at a gate: a yes-or-no condition, checked in the group chat on the Sunday. If a gate is missed, say so that day. A problem raised in week 3 is fixable; the same problem raised in week 7 is not.

WEEK	INTEGRATION OWNER	GATE — MUST BE TRUE BY SUNDAY NIGHT
1 Sep 7	Shaili	<code>Packet</code> rewritten and merged; emulator rewritten; calibration committed from WSL2; everyone's build green
2 Sep 14	Sinjan	8 MiB file transfers with a SHA-256 match at 10% loss across 10 seeds
3 Sep 21	Sohini	Go-Back-N passes integrity across 5 channel profiles × 10 seeds; first plot exists
4 Sep 28	Sohan	Core complete — all three protocols pass one shared integrity suite
5 Oct 5	Shaili	Figures 1–3 regenerated by script from committed CSVs
6 Oct 12	Sinjan	Figure 4 done; netem validation done; every member can trace a packet through code they did not write
7 Oct 19	Sohini	Full report draft read and annotated by all four
8 Oct 26	Sohan	<code>make all</code> reproduces every figure from a clean clone

Part 7 · Viva preparation

Five of the ten marks are awarded to you individually. Sir has said he will ask you to modify your code live during the demo. In week 8, run these at each other until nobody hesitates — and note that several of them are about code you did not write.

TOPIC	QUESTION
Checksum	Explain the Internet checksum byte by byte. Why one's complement? Why end-around carry? Why is a single-bit flip always caught?
Unsigned fields	Why <code>Integer.toUnsignedLong</code> on the sequence field? Show me what breaks without it.
Windows	Why does Selective Repeat need $W \leq 2^{k-1}$ when Go-Back-N only needs $W \leq 2^k - 1$? Draw the failure case.
Timers	How do you avoid a race between a timeout firing and an ACK arriving?
Karn	What is Karn's algorithm protecting against? Give a concrete example.
Loss	Walk me through what happens when an ACK is lost but the data arrived.
Reproducibility	How do you know two runs with the same seed are identical? Why <code>java.util.Random</code> ?
Measurement	How do you know your goodput curve reflects the protocol and not your machine running out of CPU?
Live change	Change the window from 32 to 8 while a transfer is running. Add a new packet type, now.

Three questions still to ask sir. The brief we have refers to a "Section 6" on individual moderation and a "Section 7" on scanning networks, but the document ends at Section 6, the proposal template — so ask whether there is a fuller version. It also states no proposal deadline and no submission method anywhere. Ask this week: five minutes in the lab is cheaper than a rejected week.